



Rapport de Projet de Fin d'Année

Tastify

Application ERP de gestion de restaurant avec analyse de sentiment

Réalisé par	Diouri Mehdi et El Kharrazi Ibtihal
Année universitaire	2025-2026
Etablissement	EMSI
Livrable	Version PDF générée depuis les sources LaTeX

Dédicaces

Nous dédions ce travail à nos familles, pour leur soutien constant, leur patience et leurs encouragements tout au long de cette année. Leur présence nous a permis de poursuivre ce projet avec sérieux, même pendant les périodes les plus chargées.

Nous le dédions également à nos enseignants, qui nous ont transmis les bases nécessaires pour concevoir, développer et défendre une solution informatique complète. Enfin, nous adressons cette dédicace à nos camarades, avec qui nous avons partagé les efforts, les difficultés et les apprentissages de ce parcours.

Remerciements

Avant de présenter notre projet, nous souhaitons exprimer notre gratitude à toutes les personnes qui ont contribué, directement ou indirectement, à sa réalisation. Nous remercions nos familles pour leur accompagnement et leur confiance.

Nous remercions également nos enseignants de l'EMSI pour les connaissances acquises en génie logiciel, développement web, bases de données, modélisation UML et gestion de projet. Ces enseignements ont constitué le socle technique et méthodologique de Tastify.

Nos remerciements s'adressent enfin à toutes les personnes qui nous ont aidés à mieux comprendre les besoins opérationnels d'un restaurant : gestion de salle, suivi de cuisine, stocks, réservations et relation client.

Résumé

Tastify est une application web full-stack destinée à faciliter la gestion quotidienne d'un restaurant. Le projet répond à plusieurs difficultés fréquemment rencontrées dans les établissements de restauration : coordination entre la salle et la cuisine, suivi des commandes, gestion des stocks, réservations, paiements, fidélisation et analyse des avis clients.

La solution repose sur un backend Django REST Framework, une base de données MySQL, Redis pour le temps réel et les tâches asynchrones, ainsi que deux applications React : une interface back-office destinée au personnel et un portail client. Tastify intègre également un Kitchen Display System permettant de suivre l'avancement des plats en cuisine, un module de paiement avec partage de l'addition et une analyse de sentiment des avis clients via l'API Hugging Face, avec un mécanisme de secours local.

Ce rapport présente le contexte du projet, l'étude de l'existant, la conception UML, l'architecture technique, les principales fonctionnalités réalisées et les tests mis en place pour valider la solution.

Abstract

Tastify is a full-stack web application designed to support the daily management of a restaurant. The project addresses common operational issues such as communication between waiters and kitchen staff, order tracking, stock management, reservations, payments, loyalty and customer feedback analysis.

The solution is built with a Django REST Framework backend, a MySQL database, Redis for real-time communication and asynchronous tasks, and two React applications: a staff back-office and a customer portal. Tastify also includes a Kitchen Display System, split-bill payment support and sentiment analysis for customer reviews using the Hugging Face API with a local fallback.

This report presents the project context, existing solutions, UML design, technical architecture, implemented features and validation tests.

Liste des abréviations

Introduction générale

La transformation numérique touche progressivement les différents secteurs économiques, y compris la restauration. Pourtant, de nombreux restaurants continuent à gérer leurs opérations avec des outils dispersés : carnets papier, feuilles de calcul, échanges verbaux entre la salle et la cuisine, ou solutions de caisse non reliées aux autres fonctions de l'établissement.

Cette organisation entraîne plusieurs limites : erreurs dans les commandes, manque de visibilité sur l'état des tables, difficulté à anticiper les besoins en ingrédients, suivi incomplet des réservations et absence d'analyse structurée des retours clients. Dans un environnement où la rapidité du service et la qualité de l'expérience client sont essentielles, ces limites peuvent avoir un impact direct sur la rentabilité et la satisfaction.

Le projet Tastify s'inscrit dans ce contexte. Il vise à proposer une application web centralisée permettant de gérer les principaux flux d'un restaurant : menu, salle, cuisine, stocks, ressources humaines, réservations, paiements, fidélité, avis clients et tableaux de bord. Le projet ne se limite pas à une simple application CRUD : il intègre du temps réel pour la coordination salle-cuisine, des tâches asynchrones pour les traitements de fond et une analyse de sentiment des commentaires clients.

Ce rapport présente d'abord le contexte général et l'étude de l'existant. Il détaille ensuite la conception fonctionnelle et UML de Tastify, puis son architecture technique et les modules réalisés. Enfin, il présente les tests mis en place et les perspectives d'amélioration.

Contexte général et étude de l'existant

Problématique

Dans un restaurant, plusieurs acteurs interviennent simultanément : le gérant, les serveurs, les cuisiniers et les clients. Chacun manipule des informations différentes, mais toutes ces informations sont liées. Une commande prise en salle concerne une table, des plats, des ingrédients, un paiement et parfois un avis client.

Lorsque ces informations sont gérées séparément, le restaurant perd en efficacité. Une rupture de stock peut être découverte trop tard, une commande peut être mal transmise, une réservation peut entrer en conflit avec une table déjà occupée, et les retours clients peuvent rester inexploités. La problématique principale est donc la suivante :

Comment concevoir une solution web centralisée, accessible et maintenable permettant de gérer les opérations d'un restaurant tout en améliorant la coordination entre les acteurs ?

Etude de l'existant

Plusieurs solutions commerciales existent pour la gestion de restaurant. Parmi les plus connues, on retrouve Lightspeed Restaurant, Toast POS, Revel Systems et L'Addition. Ces outils proposent des fonctionnalités avancées de caisse, de commande ou de reporting. Cependant, ils présentent souvent des limites pour les petites et moyennes structures : coût mensuel élevé, dépendance à du matériel spécifique, disponibilité limitée selon les pays, ou faible adaptation aux besoins locaux.

Tastify se positionne comme une solution web modulaire, open-source dans son approche technique, et conçue pour couvrir les besoins principaux sans imposer d'infrastructure lourde.

Tableau - Comparaison synthétique des solutions existantes

Objectifs du projet

Les objectifs principaux de Tastify sont les suivants :

Périmètre fonctionnel

Le périmètre retenu couvre quatre espaces d'usage :

Dans l'implémentation actuelle, ces espaces sont organisés autour de deux applications React : une application staff pour les rôles gérant, serveur et cuisinier, et une application client pour le portail public.

Analyse et conception

Acteurs du système

Tastify repose sur quatre acteurs principaux : le gérant, le serveur, le cuisinier et le client. Les droits d'accès sont contrôlés par rôle afin que chaque utilisateur accède uniquement aux fonctionnalités nécessaires.

Tableau - Acteurs et responsabilités

Diagrammes de cas d'utilisation

Les cas d'utilisation définissent les interactions attendues entre chaque acteur et le système. Les schémas ci-dessous sont conservés depuis le rapport initial.

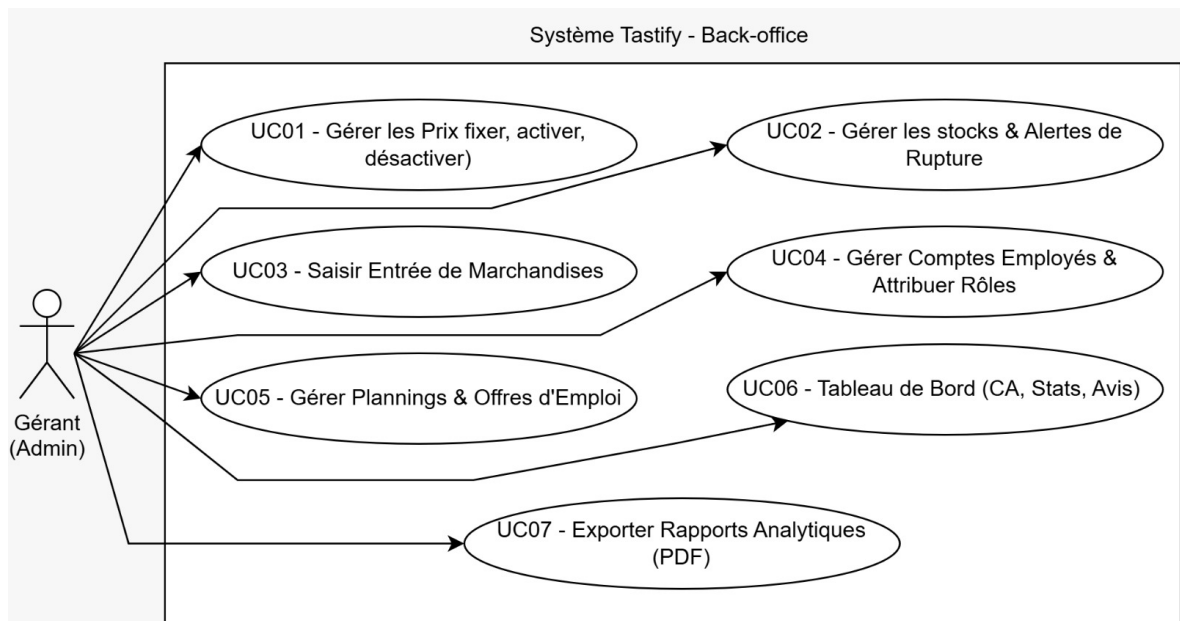


Figure - Diagramme de cas d'utilisation du module administration

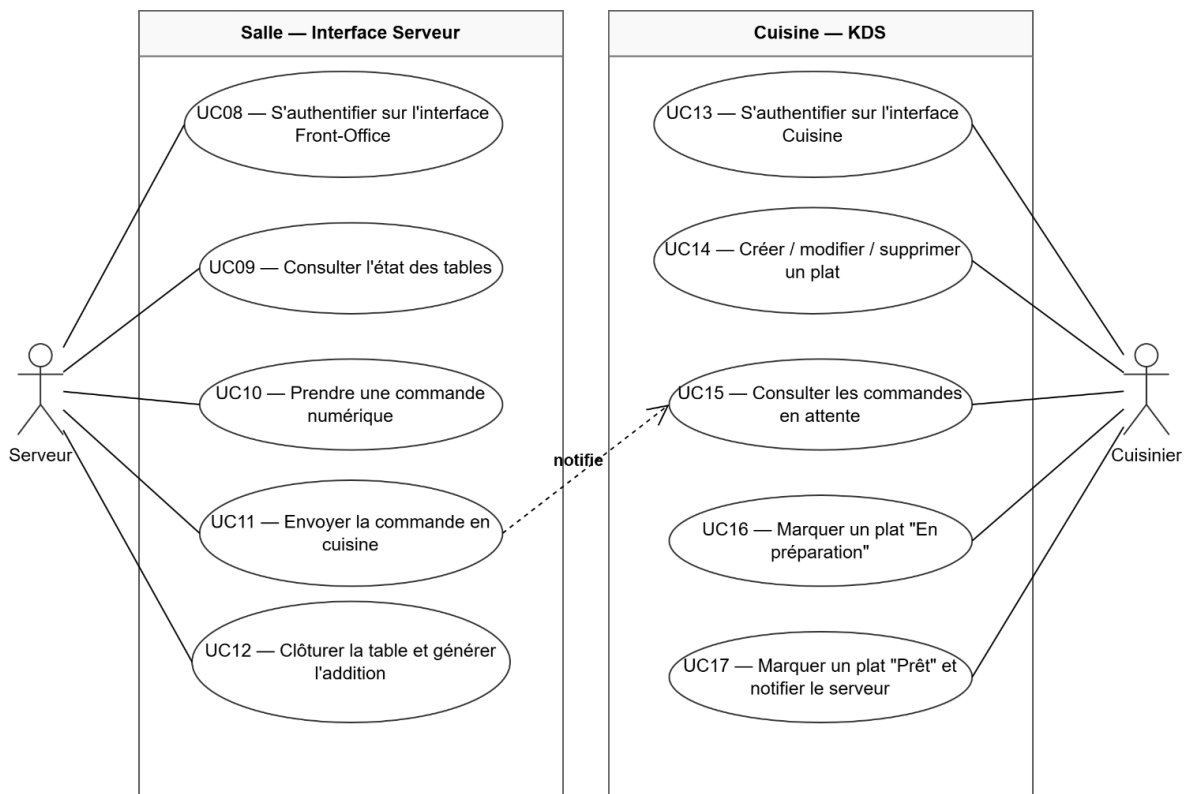


Figure - Diagramme de cas d'utilisation des modules salle et cuisine

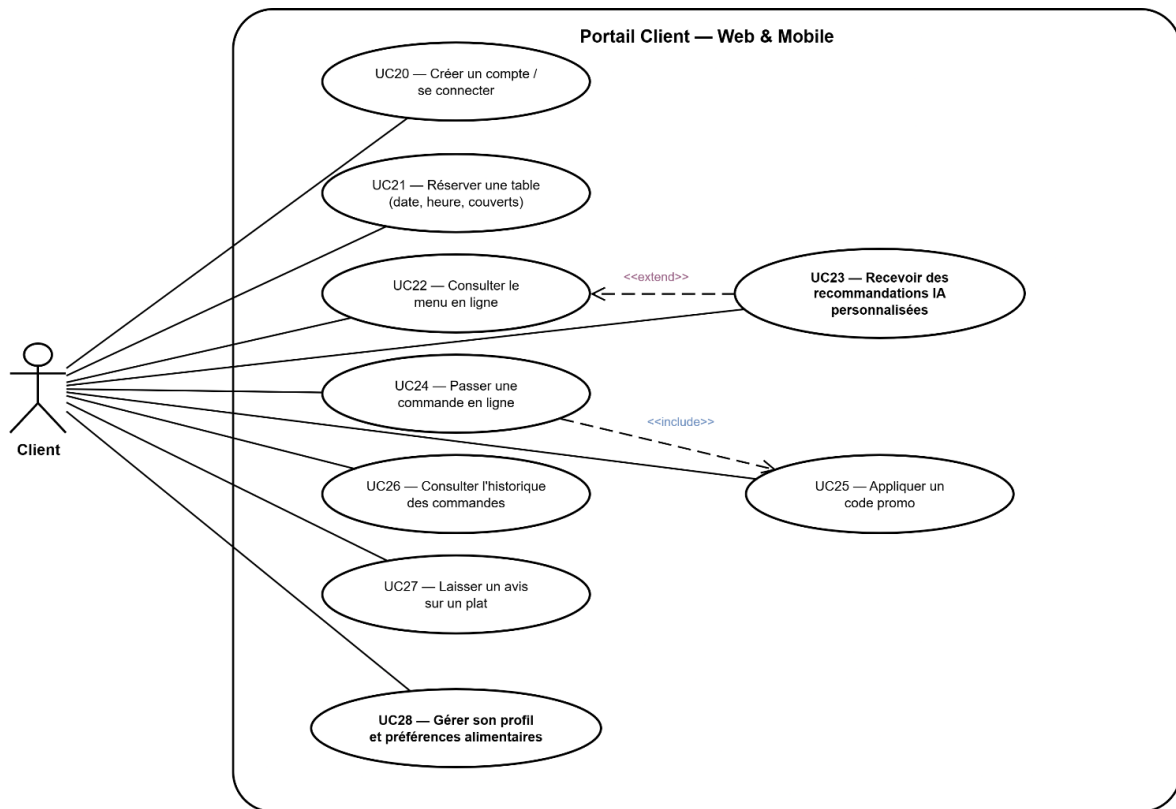


Figure - Diagramme de cas d'utilisation du portail client

Diagramme de classes

Le diagramme de classes met en évidence les entités principales : utilisateurs, plats, catégories, tables, commandes, lignes de commande, réservations, stocks, paiements, avis et éléments de fidélité. Cette modélisation est transposée en modèles Django et en tables relationnelles.

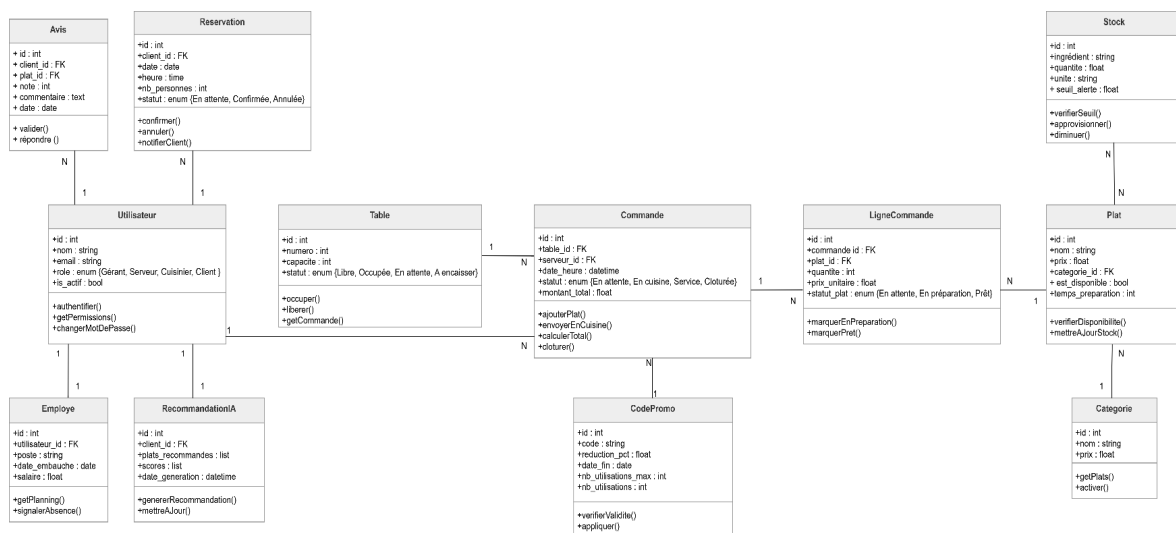


Figure - Diagramme de classes de Tastify

Principales règles métier

Les règles métier suivantes structurent le fonctionnement de l'application :

Conception des interfaces

La conception visuelle distingue les usages selon le contexte. L'interface gérant privilégie les indicateurs, les tableaux et les actions d'administration. L'interface serveur met en avant le plan de salle. L'interface cuisine adopte une vue KDS orientée production. Le portail client présente le menu et les actions publiques.

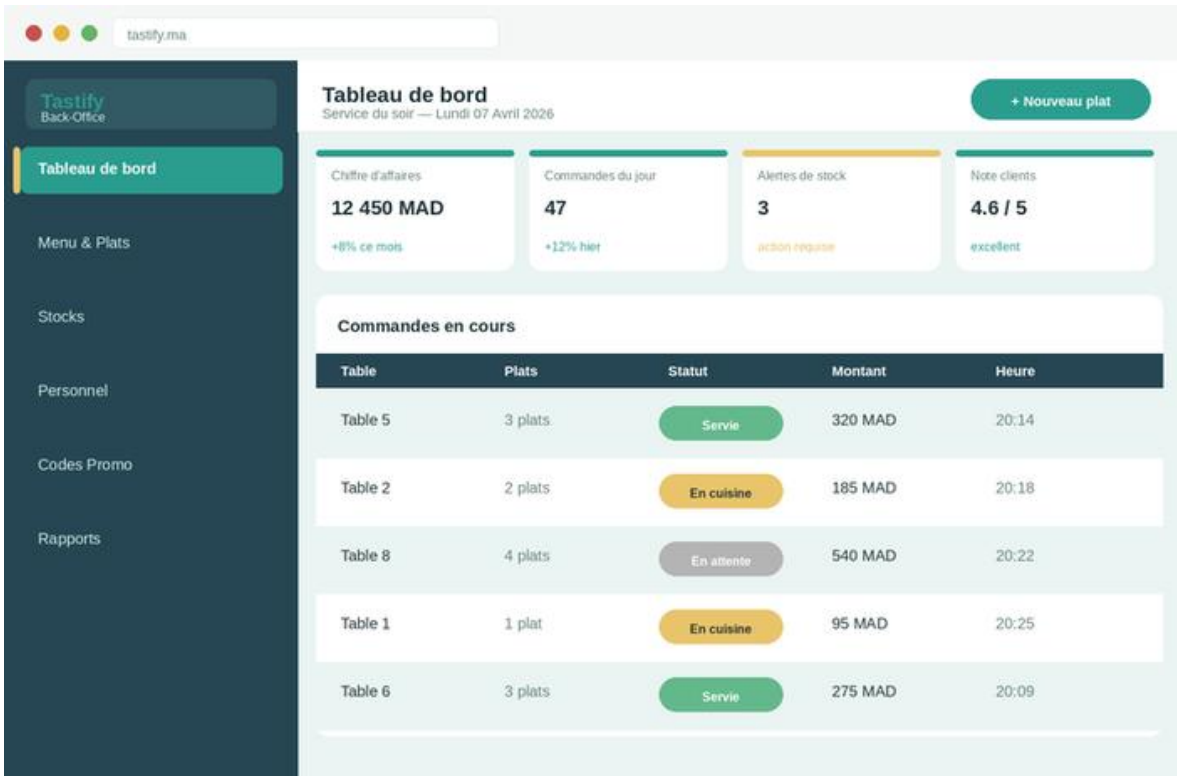


Figure - Maquette du tableau de bord gérant

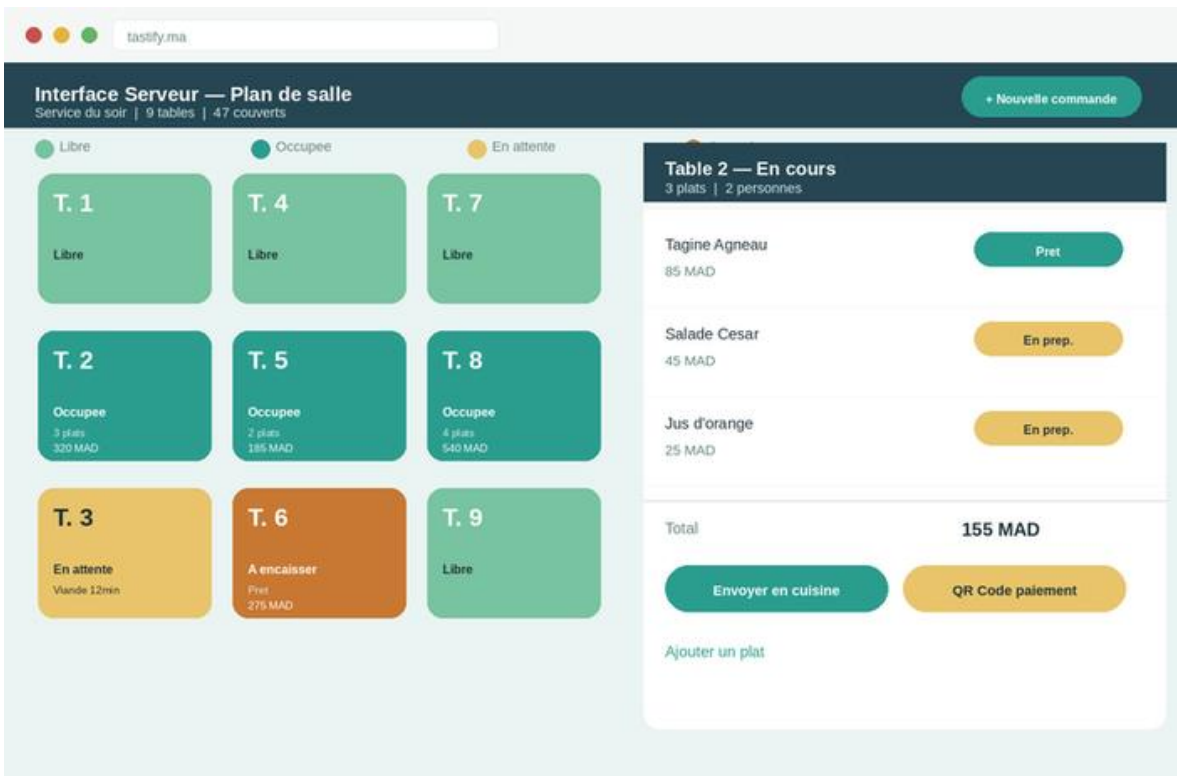


Figure - Maquette de l'interface serveur et du plan de salle

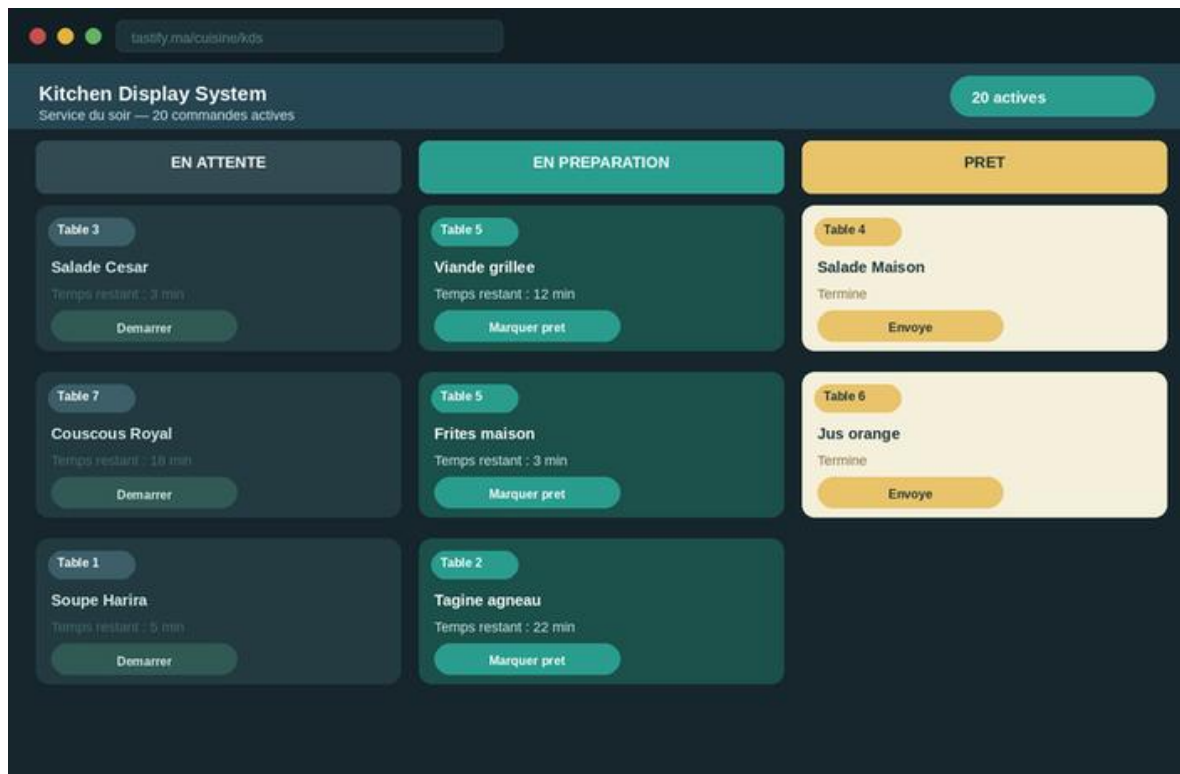


Figure - Maquette du Kitchen Display System

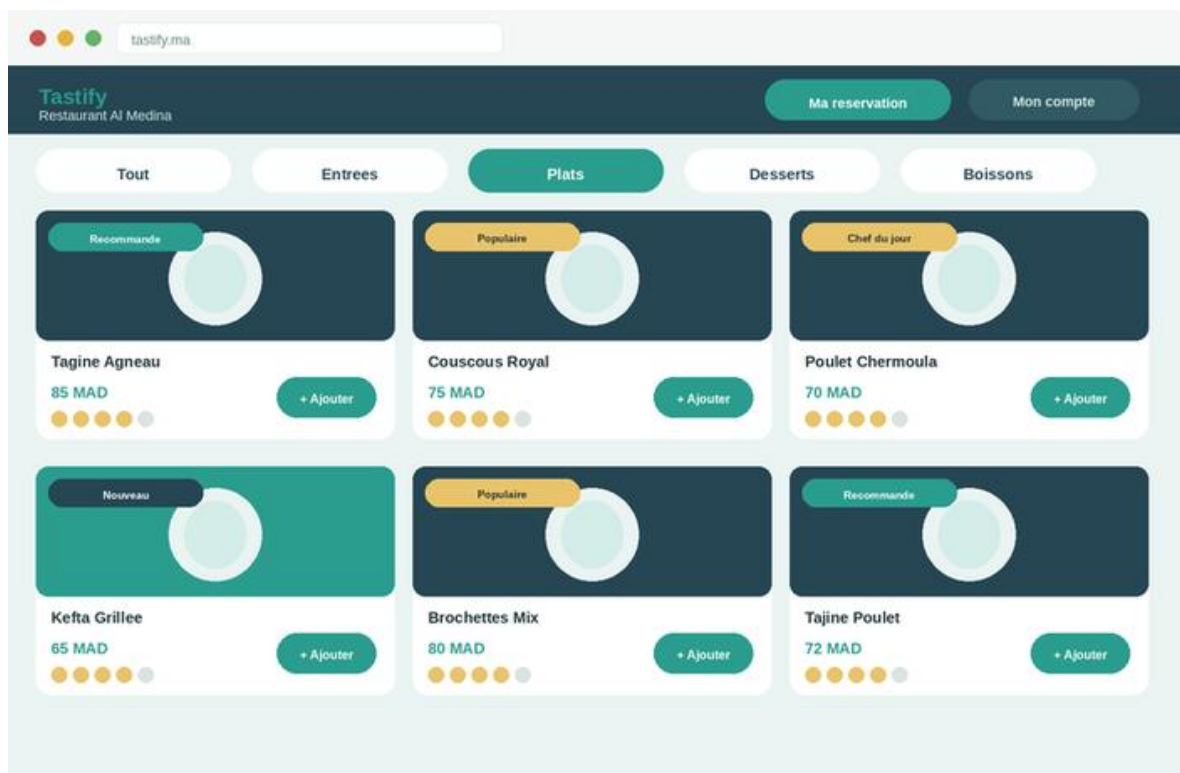


Figure - Maquette du portail client

Architecture technique et réalisation

Architecture générale

Tastify adopte une architecture web découplée. Le backend expose des API REST et des canaux WebSocket. Les interfaces frontend consomment ces services via HTTP et WebSocket.

Tableau - Stack technique réelle du projet

Backend Django

Le backend est organisé en applications Django spécialisées. Cette séparation facilite la maintenance et rend chaque domaine fonctionnel plus lisible.

Tableau - Modules backend réalisés

Authentification et sécurité

L'authentification repose sur JSON Web Token. L'access token possède une durée de vie courte, tandis que le refresh token est stocké dans un cookie HttpOnly. Dans la configuration actuelle, le refresh token expire après un jour. Les rôles applicatifs permettent de protéger les routes et les actions sensibles.

En production, le fichier de configuration active les cookies sécurisés et prévoit une configuration CORS adaptée aux domaines autorisés. Les variables sensibles sont externalisées dans l'environnement.

Frontend staff et portail client

L'application staff regroupe les espaces gérant, serveur et cuisinier. Elle utilise React Router pour organiser les routes et applique des protections selon les rôles. Le gérant accède aux modules de gestion, le serveur au plan de salle et à la prise de commande, et le cuisinier au KDS.

Le portail client est séparé dans une deuxième application React. Il couvre l'accueil, le menu, les réservations, le compte client, la fidélité, le paiement et les avis.

Les deux frontends sont configurés comme Progressive Web Apps via Vite PWA. Le cache est principalement utilisé pour les assets statiques et certaines ressources publiques ; les actions critiques restent dépendantes du backend afin d'éviter les incohérences de données.

Kitchen Display System

Le KDS permet au personnel cuisine de visualiser les commandes envoyées depuis la salle. Les lignes de commande disposent de statuts distincts : en attente, en préparation, prêt, servi ou annulé.

Un orchestrateur calcule un horaire commun de fin de préparation à partir des temps de préparation des plats. Les tâches Celery peuvent ensuite déclencher le lancement d'une ligne au bon moment. Cette logique améliore la synchronisation entre les plats d'une même commande.

Paiement et fidélité

Le module paiement gère les encaissements liés aux commandes. Il prend en charge les méthodes espèces, carte et QR code. Le partage de l'addition repose sur des contributions liées aux lignes de commande, ce qui permet à plusieurs clients de payer différentes parties de la même commande.

Le module fidélité attribue des points à un profil client. Les récompenses peuvent être configurées selon un nombre de points requis, ce qui permet d'encourager le retour des clients.

Analyse de sentiment et prévision

Le module d'avis déclenche une tâche Celery lors de la création d'un commentaire. L'analyse principale s'appuie sur l'API Hugging Face. Si le token n'est pas disponible ou si l'API ne répond pas, un mécanisme local simple sert de solution de secours. Le résultat est stocké sous forme de label positif, neutre ou négatif et d'un score associé.

La recommandation de plats reste volontairement simple dans l'état actuel : elle s'appuie principalement sur la catégorie du plat et les plats disponibles. Elle constitue une base évolutive pour une personnalisation future.

La prévision de besoins en stock repose sur la moyenne des ventes récentes. La météo peut être simulée lorsque aucune API réelle n'est configurée. Cette approche est suffisante pour un prototype académique et reste facilement explicable.

Déploiement

L'environnement Docker Compose orchestre les services suivants : base MySQL, Redis, backend Daphne, worker Celery, beat Celery, application staff et application client. Cette organisation facilite le lancement local et prépare une éventuelle mise en production sur serveur Linux.

Tests, validation et résultats

Stratégie de validation

La validation du projet repose sur plusieurs niveaux complémentaires :

Tests backend

Les tests backend couvrent notamment l'authentification, les permissions, les réservations, les paiements, le KDS, les commandes, les stocks, les utilisateurs et la configuration. Ils servent à vérifier que les règles métier sont respectées côté serveur.

Tableau - Exemples de familles de tests backend

Tests frontend et end-to-end

Les tests frontend vérifient des comportements d'interface : rendu initial, navigation, formulaires, états de chargement et persistance après rechargement. Les tests Playwright couvrent les parcours clés des deux applications React.

Tests de charge

Le projet contient une surface de commande prévue pour les tests de charge. Cependant, dans l'arborescence actuellement disponible, le fichier de campagne Locust n'est pas présent. Le rapport ne présente donc pas de résultat chiffré de charge. Cette partie peut être complétée ultérieurement si une campagne Locust est ajoutée et exécutée.

Validation fonctionnelle

La validation fonctionnelle confirme que les principaux objectifs sont couverts :

Limites

Certaines fonctionnalités restent perfectibles. La recommandation de plats n'est pas encore un modèle personnalisé complet basé sur l'historique client. La prévision de demande repose sur une moyenne simple et non sur un modèle statistique avancé. Enfin, les résultats de performance doivent être consolidés par une campagne de charge complète.

Conclusion générale

Le projet Tastify a permis de concevoir et de réaliser une application web de gestion de restaurant couvrant plusieurs besoins opérationnels : gestion du menu, plan de salle, commandes, cuisine, stocks, ressources humaines, réservations, paiements, fidélité, avis clients et tableaux de bord.

Sur le plan technique, le projet met en œuvre une architecture moderne fondée sur Django REST Framework, React, MySQL, Redis, Celery et Docker Compose. L'intégration de WebSocket améliore la coordination entre la salle et la cuisine, tandis que les tâches asynchrones permettent de traiter certains calculs ou analyses sans bloquer l'expérience utilisateur.

Le projet intègre également une première couche d'intelligence applicative : analyse de sentiment des avis clients via Hugging Face, recommandations simples de plats et estimation des besoins à partir des ventes récentes. Ces fonctionnalités constituent une base solide qui pourra être enrichie dans de futures versions.

Les perspectives d'amélioration concernent principalement la personnalisation des recommandations, l'ajout d'un vrai modèle prédictif pour les stocks, la consolidation des tests de charge, l'amélioration de l'expérience mobile et l'éventuel déploiement sur une infrastructure de production.

En conclusion, Tastify représente une solution cohérente et évolutive pour la gestion numérique d'un restaurant. Le projet nous a permis de mobiliser des compétences en conception, développement backend, développement frontend, bases de données, architecture logicielle, sécurité et validation.

Bibliographie et webographie

- Django Software Foundation. Django Documentation.
- Django REST Framework Documentation.
- React Documentation.
- MySQL 8.0 Reference Manual.
- Redis Documentation.
- Celery Documentation.
- Hugging Face Model Hub et Inference API.
- NLP Town, bert-base-multilingual-uncased-sentiment.
- Martin Fowler, Patterns of Enterprise Application Architecture.
- Booch, Rumbaugh et Jacobson, The Unified Modeling Language User Guide.